

Pronalaženje skrivenog znanja

Projektni zadatak za školsku 2025/2026. godinu

NAPOMENA

Projektni zadatak se radi samostalno. Predati projekti se upoređuju međusobno. Studenti kod kojih se utvrdi sličnost dobijaju 0 poena na projektu u ovoj školskoj godini i u ukupnom broju poena (bez obzira da li su izlazili na pismeni ispit) i nemaju prava da izlaze više na odbrane projekata u ovoj školskoj godini. Tokom odbrane projekta, student mora biti spreman da detaljno obrazloži svaki red samostalno napisanog koda za algoritme mašinskog učenja i uradi modifikaciju na licu mesta, u ograničenom vremenskom intervalu. Nerazumevanje sopstvenog koda ili neuspešna modifikacija povlači automatski diskvalifikaciju na projektu. Stoga molimo studente da samostalno rade svoje projekte, kako ne bismo bili prinuđeni da prijave šaljemo Disciplinskoj komisiji.

Projektni zadatak se sastoji iz pet celina na kojima se može ostvariti ukupno 60 poena. Zadaci se odnose na prikupljanje podataka, njihovu analizu i obradu, vizuelizaciju i implementaciju modela korišćenjem algoritama mašinskog učenja, kao i evaluaciju modela. Obavezno je uraditi bar jedan zadatak iz skupa {4 i 5} da biste ostvarili prolazan broj poena.

[10] Zadatak 1: Prikupljanje podataka

Realizovati veb indeks (eng. *web crawler/web spider*) sa veb parserom (eng. *web scraper*), koji prikuplja podatke o proizvodima bele tehnike (kućni aparati, televizori, mašine za pranje veša, frižideri, šporeti, klima uređaji i slično) sa jednog ili više od sledećih sajtova:

- <https://gigatron.rs/>
- <https://www.tehnomanija.rs/>
- <https://www.tehnimedia.rs/>
- neki drugi sajt prodavnice bele tehnike u Srbiji koji nije u ovoj listi, a ima dovoljan broj zapisa.

Formirati sopstvenu relacionu bazu podataka sa svim relevantnim informacijama o proizvodima koji se nude. Bazu realizovati kao relacionu, u tehnologiji MySQL ili PostgreSQL. Baza treba da ima najmanje 8 hiljada aktuelnih zapisa o proizvodima.

Šta je veb indeks?

Cilj veb indeksa je da se poveže na određenu veb stranu i da preuzme njen sadržaj. Parsiranjem date strane možemo naći linkove, koji vode na neke druge strane, na koje veb-indeks ponovo može da uđe i da ponovi celu proceduru. Pored otkrivanja linkova, parser može da prepozna i druge sadržaje koje veb strana ima. U vašu bazu treba da prikupite informacije o svim proizvodima - naziv proizvoda, brend, kategorija (npr. televizor, mašina

za pranje, frižider), ključne tehničke karakteristike (npr. dijagonala ekrana u inčima, kapacitet bubnja u kg, energetska klasa, snaga, zapremina frižidera u litrima), izvor prodavnice (sajt), dostupnost na lageru i cena. Za podatke koji nisu dostupni u opisu proizvoda, polja u bazi treba da ostanu prazna (NULL).

Implementaciju veb-indeksera možete raditi u programskim jezicima: C, C++, C#, Java, Python, NodeJS ili PHP. Dozvoljeno je i korišćenje i prilagođavanje neke od postojećih implementacija otvorenog koda: crawler4j, Heritrix, Nutch, Scrapy za Python, PHP-Crawler za PHP, itd.

Zbog ograničenog broja zahteva na serverima sa iste IP adrese, koristiti rotirajuće proksije ili neku drugu tehniku, kako ne biste kršili uslove korišćenja usluga izvornih veb sajtova. Prikupljanje ovih podataka koji nisu orijentisani ka ličnim podacima i koji jesu javno dostupni je dozvoljeno, ali uz molbu da broj zahteva ka serveru prilagodite, i da između zahteva bude vremenske razlike, kako domaćinu sajta ne biste izvršili Denial-of-service (DoS) napad.

Šta je veb parser?

Uloga veb parsera je da otkrije potreban sadržaj sa primljenih veb strana. Pri tome potrebno je odrediti značenje sadržaja kako bi se baza podataka popunjavala tačnim podacima. Najčešće tehnike koje se koriste pri implementaciji veb parsera su: HTML parser, DOM parser, tehnika regularnih izraza koji izdvajaju potreban sadržaj i tehnika prepoznavanja semantičkih anotacija. Za potrebe veb parsiranja takođe možete koristiti neku od postojećih implementacija (npr. biblioteka Jsoup - parsira veb stranu kao stablo elemenata, BeautifulSoup, Scrapy, ili PySpider, za Python, itd.).

Kao rezultat zadatka 1 treba da prikazete realizovanu relacionu bazu podataka popunjenu traženim podacima o proizvodima i da priložite implementacije koje su korišćene za dohvaćanje podataka. Podaci treba da budu preuzeti u konačnom vremenskom intervalu.

[10] Zadatak 2: Analiza podataka

Iz zapisa ubačenih u primarnu bazu podataka (iz zadatka 1), potrebno je izvršiti preprocesiranje podataka: odbaciti zapise koji nemaju adekvatan broj popunjenih atributa ili dopuniti polja tamo gde je to moguće. Prečišćene podatke sačuvati u novoj, revidiranoj bazi podataka (sa preostalim brojem zapisa koji ne sme biti manji od 7 hiljada) i uraditi sledeće:

- izlistati koliki je broj proizvoda po kategorijama (npr. televizori, mašine, frižideri);
- izlistati koliko proizvoda pripada najčešćim brendovima (npr. top 10 brendova po broju artikala);
- izlistati koliko proizvoda ima energetska klasu C (ili A+ po staroj klasifikaciji) ili bolju;
- izlistati koliko proizvoda ima cenu u rasponu do 50.000 dinara;
- prikazati proizvode (Top30) koji imaju:
 - kategoriju televizor,

- najveću cenu.

Kao rezultat zadatka 2 treba priložiti revidiranu i prečišćenu bazu podataka, SQL kodove realizovanih upita i generisane rezultate izvezene u eksterni format (npr. Excel ili Word dokument).

[5] Zadatak 3: Vizuelizacija podataka

Koristeći podatke iz prečišćene baze podataka (iz zadatka 2), potrebno je vizuelizovati sledeće podatke:

- Broj i procentualni odnos proizvoda za 10 najzastupljenijih kategorija.
- Broj i procentualni odnos proizvoda po brendovima (za najmanje 5 najzastupljenijih brendova).
- Broj i procentualni odnos svih proizvoda, koji po ceni pripadaju jednom od sledećih opsega:
 - manje ili jednako od 30.000 dinara,
 - između 30.001 i 100.000 dinara,
 - između 100.001 i 300.000 dinara,
 - 300.000 dinara ili više.
- Broj i procentualni odnos proizvoda po energetskej klasi.

Kao rezultat zadatka 3 treba priložiti skripte za generisanje grafika i rezultujuće grafikone izvezene u formatu slika (PNG/JPG) sa naslovom za svaki grafikon. Za vizuelizaciju se može koristiti bilo koji programski jezik, alat ili softverska biblioteka (npr. *Matplotlib/Seaborn* za Python, *ggplot2* za R, ili *Tableau/PowerBI*).

[12] Zadatak 4: Implementacija regresije

Realizovati samostalnu aplikaciju koja na osnovu prečišćenih podataka primenjuje višestruku linearnu regresiju u cilju modelovanja zavisnosti između prediktora i ciljne promenljive. Ulazni atributi (features) koje možete analizirati mogu biti: kategorija proizvoda, brend, numeričke tehničke karakteristike (npr. dijagonala ekrana, kapacitet, zapremina, snaga) i energetska klasa (kodirana numerički). Podatke podeliti na skup za treniranje i skup za testiranje, a obučavanje realizovati korišćenjem gradijentnog spusta (eng. *Gradient Descent*). Ciljna promenljiva treba da bude cena proizvoda.

U ovom zadatku nije dozvoljeno korišćenje gotovih bibliotečkih funkcija za obučavanje modela iz neke gotove biblioteke programskog jezika. Sve funkcije (funkcija troška, korak gradijentnog spusta, predikcija) moraju biti samostalno kodirane. Gotove biblioteke se mogu koristiti isključivo radi verifikacije tačnosti sopstvenih rezultata.

Aplikacija mora da poseduje korisničku formu (grafički korisnički interfejs) kroz koju korisnik unosi karakteristike uređaja, a model na izlazu ispisuje predviđenu cenu.

[11] Zadatak 5: Implementacija klasterovanja (K-means)

U okviru iste aplikacije (ili povezane aplikacije koja koristi istu bazu), primeniti algoritam K-srednjih vrednosti (*K-means*) nad prečišćenom bazom podataka, kao u zadatku 4. Pre pokretanja klasterovanja, korisnik putem grafičkog korisničkog interfejsa sam bira koje ulazne promenljive (*features*) želi da uključi u analizu, koliki je težinski udeo svake promenljive u analizi (maksimalan zbir težina po svim atributima iznosi 100%) i koliko klastera (K) želi da analizira. Izabrane promenljive moraju biti numerički pripremljene (normalizacija ili standardizacija obavezna pre klasterovanja).

Kao rezultat klasterovanja prikazati: ukupan broj proizvoda po klasteru, prosečnu cenu unutar svakog klastera i kratku tekstualnu interpretaciju karakteristika svakog klastera (npr. „jeftiniji televizori manjih dijagonala i lošijih energetskih klasa E-F-G“). Poželjno je vizuelizovati klasterove (npr. dijagram rasipanja dve izabrane dimenzije obojen po klasteru, ili bar dijagram broja proizvoda po klasteru).

Kao rezultat zadataka 4 i 5 potrebno je priložiti:

1. Izvorni programski kod realizovane aplikacije (implementaciju modela, procedure za treniranje, pomoćne klase i korisnički interfejs);
2. Kratak izveštaj (dokument) sa komentarom o realizaciji: opisati proces eksperimentisanja (koje ste parametre i arhitekture menjali, stope učenja, broj iteracija), kako ste evaluirali modele (npr. prikazati krive učenja, R^2 skor, RMSE ili *Silhouette score* za klasterove) i koji su finalni rezultati.

Napomena: Korišćenje vizuelizacije i grafikona u izveštaju za ova dva poslednja zadatka je visoko preporučeno i donosi maksimalan broj poena.

[12] Zadatak 6: Podsistem za preporučivanje (*Recommender System*)

U okviru postojeće aplikacije realizovati podsistem za preporučivanje proizvoda zasnovan na sadržaju (eng. *Content-Based Filtering*), koristeći podatke iz prečišćene baze podataka.

Funkcionalnost podsistema:

Kada korisnik u aplikaciji odabere ili pretraži jedan konkretan proizvod (npr. određeni model televizora ili veš-mašine), sistem mora da analizira njegove karakteristike i da na osnovu njih preporuči Top 5 najsliknijih proizvoda iz iste kategorije koji su dostupni u bazi.

Proces i ograničenja:

1. Kreiranje profila proizvoda (*Feature Vector*): Za računanje sličnosti između proizvoda potrebno je kombinovati numeričke attribute (cena, tehničke karakteristike) i tekstualne/kategoričke attribute (brend, naziv). Tekstualne podatke i brendove pripremiti korišćenjem tehnika kao što su *One-Hot Encoding* ili TF-IDF (eng. *Term Frequency-Inverse Document Frequency*) nad nazivom/opisom proizvoda.

2. Računanje sličnosti: Sličnost između izabranog proizvoda i ostalih artikala u bazi računati samostalno, implementacijom kosinusne sličnosti (eng. *Cosine Similarity*) ili Euklidske udaljenosti nad pripremljenim vektorima karakteristika.
3. Prikaz rezultata: Unutar korisničkog interfejsa aplikacije, nakon odabira artikla, prikazati tabelu ili listu sa 5 preporučenih proizvoda sortiranih od najslučnijeg do najmanje sličnog.

Kao rezultat zadatka 6 predati izvorni programski kod integrisan u glavnu aplikaciju (funkcije za kreiranje vektora, ručno implementiranu metriku sličnosti i logiku za sortiranje/prikaz preporuka). Izveštaj dopuniti slikovito sa prikazom barem dva primera rada sistema (npr. "Za uneti *Frižider Gorenje X*, sistem je preporučio sledećih 5 modela..."). U izveštaju ukratko prokomentarisati kako su težinski balansirane različite karakteristike (da li cena ima veći uticaj na preporuku od samog brenda i slično).

Napomena: Kao i u zadacima 4 i 5, nije dozvoljeno korišćenje gotovih bibliotečkih rešenja za računanje sličnosti (npr. *scikit-learn* funkcije za *cosine_similarity*). Formule za skalarni proizvod vektora i njihove intenzitete morate kodirati ručno.

Format predaje

Projekat se predaje kao jedna ZIP arhiva pod nazivom Indeks_Ime_Prezime.zip koja mora da sadrži sledeće foldere (direktorijume):

- /kod (posebni podfolderi za skripte za veb indeks, veb parser, i finalnu veb aplikaciju)
- /baza (SQL izvezeni fajlovi primarne i prečišćene baze)
- /izveštaj (Word/PDF dokument na nekoliko strana, koje obuhvataju analizu, grafikonske prikaze i evaluaciju modela)